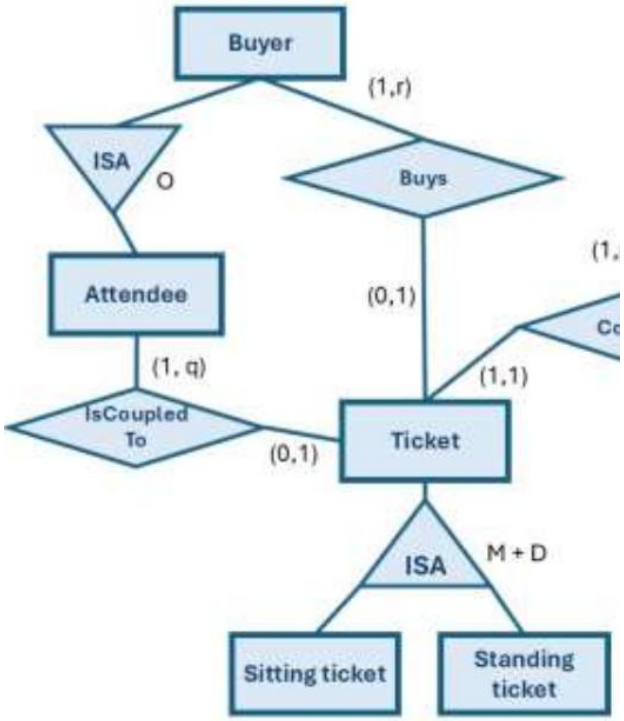
Transformation into a relational schema

<u>`Attribute`</u> means that it is a primary key
<u>`Attribute`</u> means that it is a foreign key
<u>`Attribute`</u> means that it is both

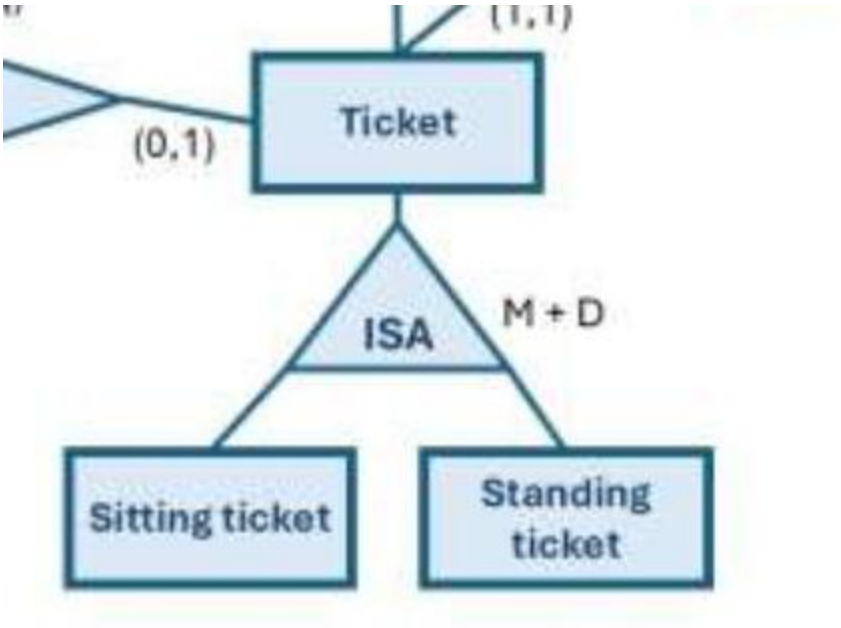
Converting entities to relational schema

(1) `Buyer` and `Attendee`



A `Buyer` is inherently an `Attendee` hence:
`Attendee(`<u>`AttendeeId`</u>`, Name, Address)`
`Buyer(`<u>`AttendeeId`</u>`, PaymentMethod, Email, Password)`
The primary key in `Buyer` is both a primary key and foreign key, enforcing the `IsA` relationship.

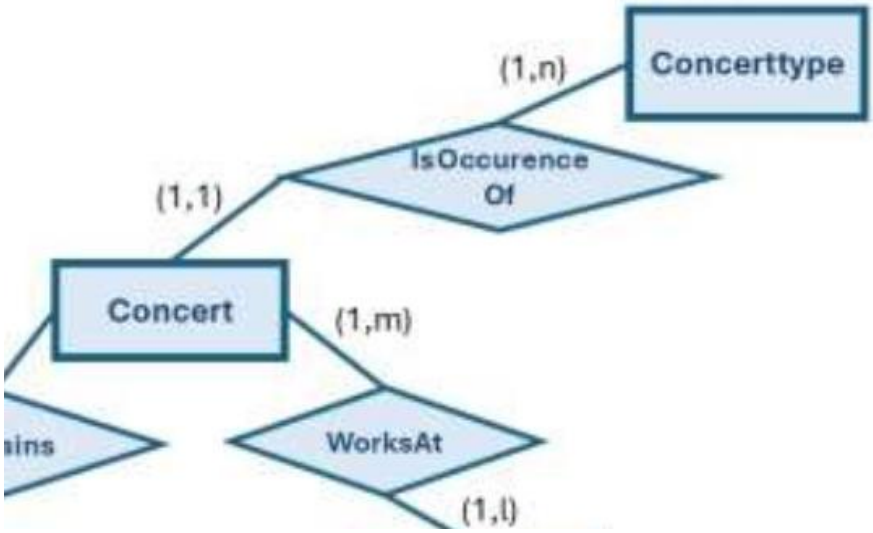
(2) `Ticket`, `SittingTicket` and `StandingTicket`



(3) To mimic the same `IsA` relationship, we do the same for `Ticket`, `SittingTicket` and `StandingTicket`.

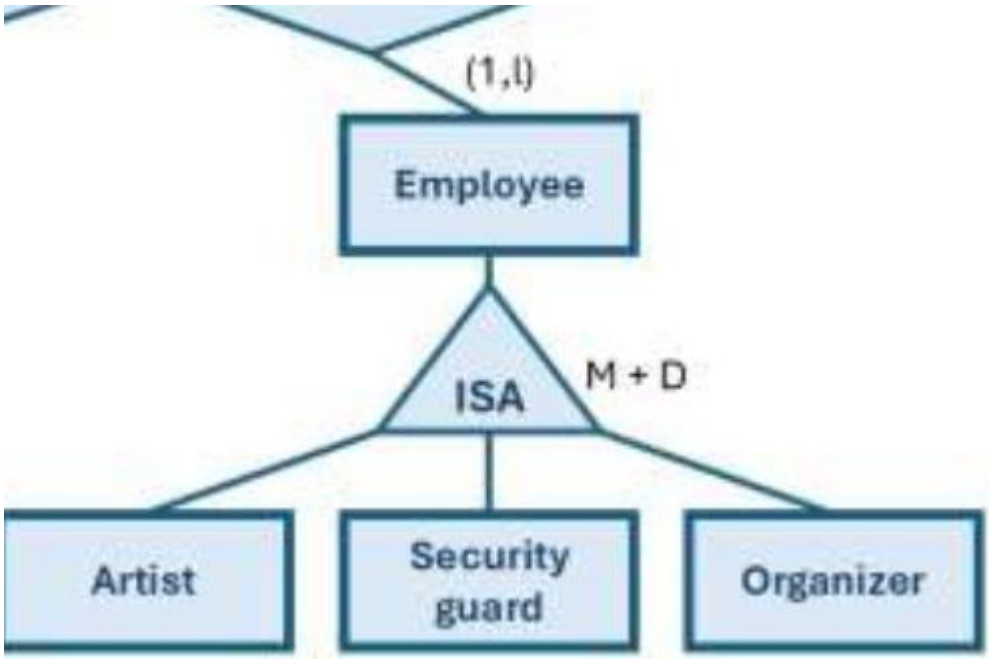
`Ticket(TicketId, Price)`
`SittingTicket(TicketId, SeatNr)`
`StandingTicket(TicketId, RegionNr)`

(4) `ConcertType` and `Concert`



`ConcertType(TypeId, Title)`
`Concert(ConcertId, Date, StartTime, Duration, Location, Capacity)`

(5) `Employee`, `Artist`, `SecurityGuard` and `Organizer`



To mimic the same `IsA` relationship, we do the same for `Employee`, `Artist`, `SecurityGuard` and `Organizer`.

`Employee(EmployeeID, Name, Address, Salary)`
`Artist(EmployeeId, Genre, Popularity)`
`SecurityGuard(EmployeeId, Rank)`
`Organizer(EmployeeId, Specialization)`

Adding constraints and DDL statements

Each `Attendee` must have a `Ticket`, therefore we add a foreign key with a **UNIQUE NOT NULL** constraint to mimic a (1, 1) relationship. Thus, the new schema becomes:

`Attendee(AttendeeId, TicketId, Name, Address)`

```sql
CREATE TABLE Attendee (
    AttendeeId INTEGER,
    TicketId INTEGER UNIQUE NOT NULL,
    Name TEXT NOT NULL,
    Address TEXT NOT NULL,
    PRIMARY KEY (AttendeeId),
    FOREIGN KEY (TicketId) REFERENCES Ticket(TicketId)
);

CREATE TABLE Buyer (
    AttendeeId INTEGER,
    EMail TEXT NOT NULL,
    Password TEXT NOT NULL,
    PaymentMethod TEXT NOT NULL,
    PRIMARY KEY (AttendeeId),
    FOREIGN KEY (AttendeeId) REFERENCES Attendee(AttendeeId)
);
```

Each `Concert` must have a `ConcertType`, therefore we add a foreign key and to enforce the (1, n) relationship between the two, we use a **NOT NULL** constraint to `Concert`. Thus, the new schema becomes: `Concert(ConcertId, TypeId, Date, StartTime, Duration, Location, Capacity)`

```sql
CREATE TABLE ConcertType (
    TypeId INTEGER,
    Title TEXT NOT NULL,
    PRIMARY KEY (TypeID)
);

CREATE TABLE Concert (
    ConcertId INTEGER,
    TypeID INTEGER NOT NULL,
    Date TEXT,
    StartTime TEXT,
    Duration REAL,
    Location TEXT NOT NULL,
    Capacity INTEGER NOT NULL,
    PRIMARY KEY (ConcertId),
    FOREIGN KEY (TypeID) REFERENCES ConcertType(TypeId)
);
```

Each `Ticket` is also for a `Concert`, therefore we add a foreign key and to enforce the (n, 1) relationship we add a **NOT NULL** constraint to `Ticket`. Thus, the new schema becomes:

`Ticket(TicketId, ConcertId, Price)`

```sql
CREATE TABLE Ticket (
    TicketId INTEGER,
    ConcertId INTEGER NOT NULL,
    Price Integer NOT NULL,
    PRIMARY KEY (TicketId),
    FOREIGN KEY (ConcertId) REFERENCES Concert(ConcertId)
);

CREATE TABLE SittingTicket (
    TicketId INTEGER,
    SeatNr INTEGER NOT NULL,
    PRIMARY KEY (TicketId),
    FOREIGN KEY (TicketId) REFERENCES Ticket(TicketId)
);

CREATE TABLE StandingTicket (
    TicketId INTEGER,
    RegionNr INTEGER NOT NULL,
    PRIMARY KEY (TicketId),
    FOREIGN KEY (TicketId) REFERENCES Ticket(TicketId)
);
```

Each `Employee` works at a `Concert` with a (1, n) relationship; therefore, we add a foreign key with a **UNIQUE NOT NULL** constraint. The new schema becomes:

`Employee(EmployeeId, ConcertId, Name, Address, Salary)`

```sql
CREATE TABLE Employee (
    EmployeeId INTEGER,
    ConcertId INTEGER UNIQUE NOT NULL,
    Name TEXT NOT NULL,
    Address TEXT NOT NULL,
    Salary INTEGER NOT NULL,
    PRIMARY KEY (EmployeeId),
    FOREIGN KEY (ConcertId) REFERENCES Concert(ConcertId)
);

CREATE TABLE Artist(
    EmployeeId INTEGER,
    Genre TEXT,
    Popularity REAL,
    PRIMARY KEY (EmployeeId),
    FOREIGN KEY (EmployeeId) REFERENCES Employee(EmployeeId)
);

CREATE TABLE SecurityGuard (
    EmployeeId INTEGER,
    Rank TEXT NOT NULL,
    PRIMARY KEY (EmployeeId),
    FOREIGN KEY (EmployeeId) REFERENCES Employee(EmployeeId)
);

CREATE TABLE Organizer (
    EmployeeId INTEGER,
    Specialization TEXT NOT NULL,
    PRIMARY KEY (EmployeeId),
    FOREIGN KEY (EmployeeId) REFERENCES Employee(EmployeeId)
);
```